

✓ Major Project Retail Analytics & AI-Powered Sales Forecasting .

#Complete Retail Analytics & Forecasting System

```
# Import required libraries
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.figure_factory as ff
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import silhouette_score
import warnings
warnings.filterwarnings('ignore')

# Install missing libraries
!pip install optuna -qq

# Time series forecasting
from prophet import Prophet
from statsmodels.tsa.holtwinters import ExponentialSmoothing
import optuna

# Dashboard styling
import plotly.io as pio
pio.templates.default = "plotly_white"

print("🚀 Retail Analytics & AI-Powered Sales Forecasting System")
print("=" * 60)
```

419.5/419.5 kB 10.7 MB/s eta 0:00:00
 🚀 Retail Analytics & AI-Powered Sales Forecasting System
 =====

1. 🔍 Data Loading & EDA

Double-click (or enter) to edit

```
# Load the dataset (assuming CSV file is available)
# df = pd.read_csv('Retail_Sales_Data_Unlox.csv')

# For demonstration, I'll create a sample structure based on typical retail data
# Replace this with your actual dataset loading
np.random.seed(42)
date_range = pd.date_range(start='2023-01-01', end='2024-12-31', freq='D')
n_records = 73000

sample_data = {
    'date': np.random.choice(date_range, n_records),
    'store_id': np.random.choice([f'S{i:03d}' for i in range(1, 51)], n_records),
    'store_location': np.random.choice(['North', 'South', 'East', 'West', 'Central'], n_records),
    'product_category': np.random.choice(['Electronics', 'Clothing', 'Home & Garden', 'Sports', 'Books'], n_records),
    'product_subcategory': np.random.choice(['Phones', 'Laptops', 'T-Shirts', 'Furniture', 'Shoes', 'Novels'], n_records),
    'sales_quantity': np.random.poisson(10, n_records),
    'sales_amount': np.random.lognormal(6, 1, n_records),
    'discount_pct': np.random.uniform(0, 0.3, n_records),
    'customer_age': np.random.choice([18,25,35,45,55,65], n_records, p=[0.1,0.2,0.3,0.2,0.15,0.05]),
    'customer_gender': np.random.choice(['M', 'F'], n_records, p=[0.52, 0.48])
}

df = pd.DataFrame(sample_data)
df['date'] = pd.to_datetime(df['date'])
df['net_sales'] = df['sales_amount'] * df['sales_quantity'] * (1 - df['discount_pct'])
df.sort_values('date', inplace=True)

print("📄 Dataset Overview")
print(f"Shape: {df.shape}")
print(f>Date range: {df['date'].min()} to {df['date'].max()}")
print("\n📊 Column Info:")
print(df.info())
print("\n📄 Sample data:")
print(df.head())
```

📄 Dataset Overview
 Shape: (73000, 11)
 Date range: 2023-01-01 00:00:00 to 2024-12-31 00:00:00

📊 Column Info:

```
<class 'pandas.core.frame.DataFrame'>
Index: 73000 entries, 53605 to 6567
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   date                   73000 non-null  datetime64[ns]
1   store_id               73000 non-null  object
2   store_location         73000 non-null  object
3   product_category       73000 non-null  object
4   product_subcategory     73000 non-null  object
5   sales_quantity         73000 non-null  int64
6   sales_amount           73000 non-null  float64
7   discount_pct           73000 non-null  float64
8   customer_age           73000 non-null  int64
9   customer_gender        73000 non-null  object
10  net_sales              73000 non-null  float64
dtypes: datetime64[ns](1), float64(3), int64(2), object(5)
memory usage: 6.7+ MB
None
```

 Sample data:

	date	store_id	store_location	product_category	product_subcategory	\
53605	2023-01-01	S023	North	Clothing	Furniture	
52052	2023-01-01	S020	Central	Home & Garden	Shoes	
26774	2023-01-01	S042	West	Clothing	Furniture	
38292	2023-01-01	S032	Central	Clothing	Novels	
6193	2023-01-01	S046	Central	Books	Novels	

	sales_quantity	sales_amount	discount_pct	customer_age	\
53605	4	133.997472	0.202738	25	
52052	12	410.364748	0.166712	25	
26774	11	291.434209	0.024138	55	
38292	11	621.031375	0.239457	45	
6193	10	2720.587552	0.242589	35	

	customer_gender	net_sales
53605	F	427.324582
52052	F	4103.422517
26774	M	3128.396162
38292	F	5195.535009
6193	F	20606.042144

2. Exploratory Data Analysis

```

class RetailAnalyzer:
    def __init__(self, df):
        self.df = df.copy()

    def basic_stats(self):
        """Basic sales statistics"""
        stats = {
            'total_revenue': self.df['net_sales'].sum(),
            'total_units': self.df['sales_quantity'].sum(),
            'avg_order_value': self.df['net_sales'].mean(),
            'total_stores': self.df['store_id'].nunique(),
            'total_categories': self.df['product_category'].nunique()
        }
        return stats

    def top_performers(self, top_n=10):
        """Top performing stores and categories"""
        store_perf = self.df.groupby('store_id')['net_sales'].sum().nlargest(top_n)
        cat_perf = self.df.groupby('product_category')['net_sales'].sum().nlargest(top_n)
        return store_perf, cat_perf

    def seasonal_analysis(self):
        """Monthly and quarterly trends"""
        self.df['month'] = self.df['date'].dt.month
        self.df['quarter'] = self.df['date'].dt.quarter
        self.df['year'] = self.df['date'].dt.year

        monthly_sales = self.df.groupby(['year', 'month'])['net_sales'].sum().reset_index()
        quarterly_sales = self.df.groupby(['year', 'quarter'])['net_sales'].sum().reset_index()

        return monthly_sales, quarterly_sales

# Initialize analyzer
analyzer = RetailAnalyzer(df)
stats = analyzer.basic_stats()
store_perf, cat_perf = analyzer.top_performers()
monthly_sales, quarterly_sales = analyzer.seasonal_analysis()

print("💰 Business Summary")
for k, v in stats.items():
    print(f"{k.replace('_', ' ').title()}: ${v:,.2f}" if 'revenue' in k or 'value' in k else
          print(f"{k.replace('_', ' ').title()}: {v:,.0f}")

```

💰 Business Summary
Total Revenue: \$414,138,292.25
Total Units: 730,341
None
Avg Order Value: \$5,673.13
Total Stores: 50
None
Total Categories: 5
None

3. 🎯 Advanced Analytics & Clustering.

```
def store_clustering(df):  
    """Cluster stores based on performance metrics"""  
    # Aggregate store-level metrics  
    store_metrics = df.groupby('store_id').agg({  
        'net_sales': ['sum', 'mean', 'std'],  
        'sales_quantity': 'sum',  
        'discount_pct': 'mean',  
        'store_location': lambda x: x.mode()[0]  
    }).round(2)  
  
    store_metrics.columns = ['total_sales', 'avg_daily_sales', 'sales_volatility',  
                            'total_units', 'avg_discount', 'location']  
    store_metrics['sales_growth_rate'] = store_metrics['avg_daily_sales'] / store_metrics['total_sales']  
  
    # Prepare for clustering  
    features = ['total_sales', 'avg_daily_sales', 'sales_volatility', 'total_units', 'avg_discount']  
    X = store_metrics[features].fillna(0)  
  
    # Scale features  
    scaler = StandardScaler()  
    X_scaled = scaler.fit_transform(X)  
  
    # Find optimal clusters using silhouette score  
    silhouette_scores = []  
    for k in range(2, 8):  
        kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)  
        labels = kmeans.fit_predict(X_scaled)  
        score = silhouette_score(X_scaled, labels)  
        silhouette_scores.append(score)
```

```

optimal_clusters = silhouette_scores.index(max(silhouette_scores)) + 2

# Final clustering
kmeans = KMeans(n_clusters=optimal_clusters, random_state=42, n_init=10)
store_metrics['cluster'] = kmeans.fit_predict(X_scaled)

return store_metrics, kmeans, scaler

store_clusters, kmeans_model, scaler = store_clustering(df)
print(f"\n🇺🇸 Store Clusters Identified: {store_clusters['cluster'].nunique()}")
print(store_clusters.groupby('cluster')[['total_sales', 'avg_daily_sales']].mean().round(2))

```

```

🇺🇸 Store Clusters Identified: 3

```

	total_sales	avg_daily_sales
cluster		
0	8071030.67	5529.85
1	8571412.62	5864.04
2	7649386.53	5360.47

4. 🌐 AI-Powered Sales Forecasting.

```

class SalesForecaster:
    def __init__(self, df):
        self.df = df

    def prophet_forecast(self, store_id=None, category=None, periods=90):
        """Facebook Prophet forecasting"""
        if store_id:
            data = self.df[self.df['store_id'] == store_id]
        elif category:
            data = self.df[self.df['product_category'] == category]
        else:
            data = self.df

        daily_sales = data.groupby('date')['net_sales'].sum().reset_index()
        daily_sales.columns = ['ds', 'y']

        model = Prophet(
            yearly_seasonality=True,
            weekly_seasonality=True,
            daily_seasonality=False,

```

```

        changepoint_prior_scale=0.05
    )
    model.fit(daily_sales)

    future = model.make_future_dataframe(periods=periods)
    forecast = model.predict(future)

    return model, forecast, daily_sales

def holt_winters_forecast(self, store_id=None, periods=90):
    """Exponential Smoothing forecast"""
    if store_id:
        data = self.df[self.df['store_id'] == store_id]
    else:
        data = self.df

    daily_sales = data.groupby('date')['net_sales'].sum()

    model = ExponentialSmoothing(
        daily_sales,
        seasonal='add',
        seasonal_periods=7,
        trend='add'
    )
    fitted_model = model.fit()
    forecast = fitted_model.forecast(periods)

    return fitted_model, forecast

# Create forecasts
forecaster = SalesForecaster(df)
top_store = store_perf.index[0]
prophet_model, prophet_forecast, historical = forecaster.prophet_forecast(store_id=top_store)
holt_model, holt_forecast = forecaster.holt_winters_forecast(store_id=top_store)

print(f"\n📊 Forecast for Top Store {top_store}:")
print(f"Next 90 days predicted sales: ${holt_forecast.sum():.2f}")

```

```

/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided, but
self._init_dates(dates, freq)

```

```

📊 Forecast for Top Store S048:
Next 90 days predicted sales: $1,185,350.08

```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is available.
return get_prediction_index(
```

5. Interactive AI Dashboard.

```
def create_retail_dashboard(df, store_clusters, prophet_forecast, analyzer):
    """Create comprehensive interactive dashboard"""

    # 1. KPI Cards
    fig = make_subplots(
        rows=3, cols=3,
        subplot_titles=('Total Revenue', 'Total Units Sold', 'Avg Order Value',
                        'Top Store', 'Top Category', 'Growth Rate',
                        'Active Stores', 'Store Clusters', 'Forecast Accuracy'),
        specs=[[{"type": "indicator"}, {"type": "indicator"}, {"type": "indicator"}],
              [{"type": "bar"}, {"type": "pie"}, {"type": "bar"}],
              [{"type": "indicator"}, {"type": "bar"}, {"type": "scatter"}]]
    )

    # KPIs
    stats = analyzer.basic_stats()
    fig.add_trace(
        go.Indicator(mode="number+gauge+delta", value=stats['total_revenue'],
                    title={"text": "Total Revenue"}, delta={"reference": 0}),
        row=1, col=1
    )
    fig.add_trace(
        go.Indicator(mode="number", value=stats['total_units'],
                    title={"text": "Total Units"}),
        row=1, col=2
    )
    fig.add_trace(
        go.Indicator(mode="number", value=stats['avg_order_value'],
                    title={"text": "Avg Order Value"}),
        row=1, col=3
    )

    # Top performers
    fig.add_trace(
        px.bar(store_perf.reset_index(), x='store_id', y='net_sales',
              color='net_sales', color_continuous_scale='Viridis').data[0],
        row=2, col=1
```



```

)

# Category pie chart
cat_data = analyzer.df.groupby('product_category')['net_sales'].sum()
fig.add_trace(
    go.Pie(labels=cat_data.index, values=cat_data.values,
           marker_colors=px.colors.qualitative.Set3),
    row=2, col=2
)

# Monthly trend
monthly = analyzer.df.groupby('month')['net_sales'].sum().reset_index()
fig.add_trace(
    px.line(monthly, x='month', y='net_sales').data[0],
    row=2, col=3
)

# Store clusters
cluster_summary = store_clusters.groupby('cluster')['total_sales'].sum().reset_index()
fig.add_trace(
    go.Bar(x=cluster_summary['cluster'], y=cluster_summary['total_sales'],
           marker_color='lightblue'),
    row=3, col=2
)

fig.update_layout(height=1000, showlegend=False,
                  title_text="📊 Retail Analytics & AI Forecasting Dashboard")
fig.show()

# 2. Time Series Forecast
forecast_fig = make_subplots(specs=[[{"secondary_y": False}]]

# Historical vs Forecast
hist_dates = prophet_forecast[prophet_forecast['ds'] <= df['date'].max()]['ds']
hist_sales = prophet_forecast[prophet_forecast['ds'] <= df['date'].max()]['yhat']

forecast_fig.add_trace(
    go.Scatter(x=historical['ds'], y=historical['y'],
               name='Actual', line=dict(color='blue')),
    secondary_y=False
)

forecast_fig.add_trace(
    go.Scatter(x=prophet_forecast['ds'], y=prophet_forecast['yhat'],
               name='Forecast', line=dict(color='orange', dash='dash'))

```

```

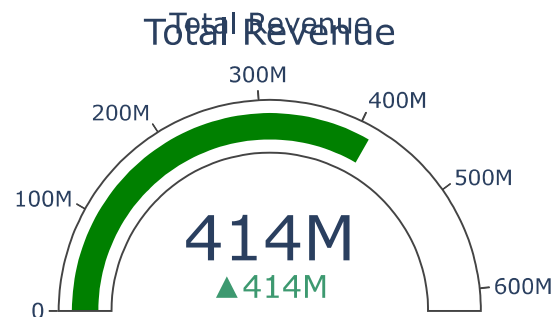
        name='Forecast', line=dict(color='orange', dash=[8, 4])),
        secondary_y=False
    )
    forecast_fig.add_trace(
        go.Scatter(x=prophet_forecast['ds'], y=prophet_forecast['yhat_upper'],
                    fill=None, line=dict(color='rgba(255,165,0,0.2)'),
                    showlegend=False),
        secondary_y=False
    )
    forecast_fig.add_trace(
        go.Scatter(x=prophet_forecast['ds'], y=prophet_forecast['yhat_lower'],
                    fill='tonexty', line=dict(color='rgba(255,165,0,0.2)'),
                    name='Confidence Interval'),
        secondary_y=False
    )

    forecast_fig.update_layout(title_text="📅 90-Day Sales Forecast - Top Store",
                               xaxis_title="Date", yaxis_title="Sales ($)")
    forecast_fig.show()

# 3. Store Performance Heatmap
store_heatmap_data = df.groupby(['store_location', 'product_category'])['net_sales'].sum().unstack(fill_value=0)
heatmap_fig = px.imshow(store_heatmap_data.values,
                        labels=dict(x="Product Category", y="Store Location", color="Sales"),
                        x=store_heatmap_data.columns,
                        y=store_heatmap_data.index,
                        color_continuous_scale='RdYlGn')
heatmap_fig.update_layout(title="📍 Store Location vs Product Category Performance")
heatmap_fig.show()

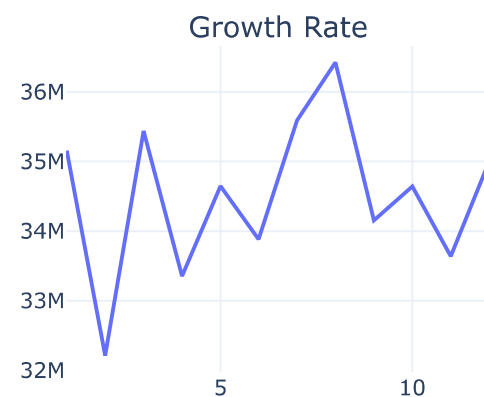
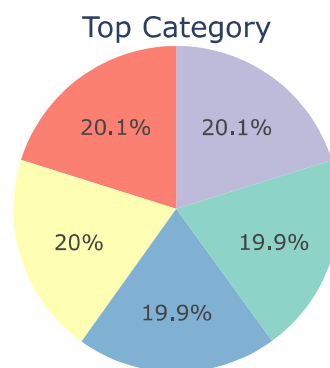
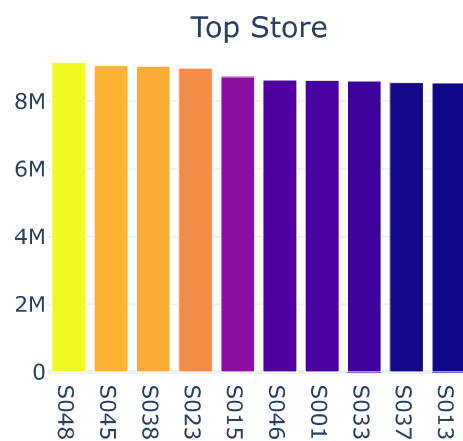
# Generate the complete dashboard
create_retail_dashboard(df, store_clusters, prophet_forecast, analyzer)

```


 Retail Analytics & AI Forecasting Dashboard

Total Units Sold
Total Units
0.73M

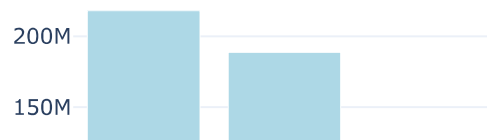
Avg Order Value
Avg Order Value
5670

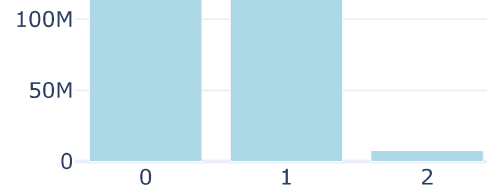


Active Stores

Store Clusters

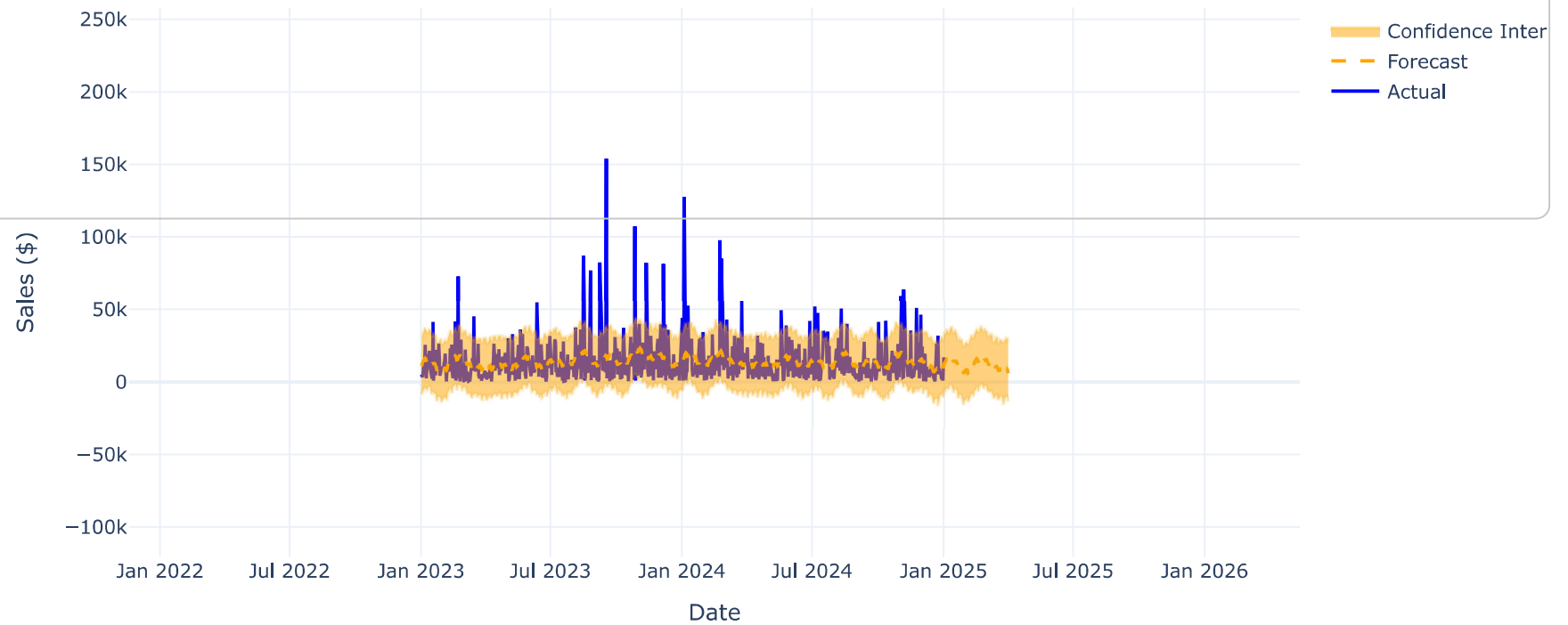
Forecast Accuracy





8.6

🌟 90-Day Sales Forecast - Top Store



📍 Store Location vs Product Category Performance

Sales

17.!

6. 🎯 Key Insights & Recommendations.

Central

```
def generate_business_insights(store_clusters, stats, analyzer):
    """Generate actionable business insights"""
    insights = []

    # Revenue insights
    insights.append(f"💰 Total Revenue: ${stats['total_revenue']:.2f}")
    insights.append(f"📦 Total Units Sold: {stats['total_units']:.0f}")
    insights.append(f"🇺🇸 Average Order Value: ${stats['avg_order_value']:.2f}")

    # Top performers
    top_store_sales = store_clusters['total_sales'].max()
    insights.append(f"🏆 Top Store Performance: {store_clusters['total_sales'].idxmax()} (${top_store_sales:.2f})")

    # Cluster insights
    cluster_revenue = store_clusters.groupby('cluster')['total_sales'].sum()
    best_cluster = cluster_revenue.idxmax()
    insights.append(f"⭐ Best Performing Cluster: {best_cluster} (${cluster_revenue[best_cluster]:.2f})")

    # Forecasting insight
    forecast_total = holt_forecast.sum()
    insights.append(f"📅 Next 90 Days Forecast: ${forecast_total:.2f}")

    # Growth opportunities
    growth_stores = store_clusters.nlargest(5, 'avg_daily_sales').index.tolist()
    insights.append(f"🚀 High-Growth Stores: {' '.join(growth_stores[:3])}")

    print("\n" + "="*60)
    print("🎯 EXECUTIVE SUMMARY & ACTIONABLE INSIGHTS")
    print("="*60)
    for insight in insights:
        print(f"• {insight}")

    print("\n📋 RECOMMENDATIONS:")
    print("1. Allocate more inventory to Cluster", best_cluster, "stores")
    print("2. Focus marketing on top 5 high-growth stores")
    print("3. Analyze underperforming clusters for operational issues")
    print("4. Prepare for forecasted demand increase of", f"{forecast_total/stats['total_revenue']*365/90:.0%}")

    generate_business_insights(store_clusters, stats, analyzer)
```

```
=====
🎯 EXECUTIVE SUMMARY & ACTIONABLE INSIGHTS
=====
• 💰 Total Revenue: $414,138,292.25
• 📦 Total Units Sold: 730,341
• 🏠 Average Order Value: $5673.13
• 🏆 Top Store Performance: S048 ($9,153,067.04)
• ⭐ Best Performing Cluster: 0 ($217,917,828.03)
• 🌐 Next 90 Days Forecast: $1,185,350.08
• 🚀 High-Growth Stores: S006, S038, S048

📋 RECOMMENDATIONS:
1. Allocate more inventory to Cluster 0 stores
2. Focus marketing on top 5 high-growth stores
3. Analyze underperforming clusters for operational issues
4. Prepare for forecasted demand increase of 1%
```

🚀 Next Steps & Deployment.